

一、整合概述

OpenSSH 是 SSH (Secure Shell) 协议的开源实现，提供加密的远程登录、文件传输和隧道功能。本文档记录了 OpenSSH 在 OpenHarmony hispark_hi3519dv500_linux 产品中的整合过程。

版本信息：

- OpenSSH 版本：9.9p1
- OpenSSL 版本：3.0.7 (运行时共享库)
- zlib 版本：1.2.13
- OpenHarmony 版本：4.0-Release
- 目标产品：hispark_hi3519dv500_linux
- 目标架构：ARM64 (aarch64)
- 编译日期：2026-05-18

二、整合步骤

1. 产品配置修改

文件： `vendor/hisilicon/hispark_hi3519dv500_linux/config.json`

在 `thirdparty` 子系统中添加 `zlib` 和 `openssl` 组件 (`openssh` 的依赖)，在 `developtools` 子系统中添加 `openssh` 组件：

```
{
  "subsystem": "thirdparty",
  "components": [
    { "component": "iptables", "features": [ "" ] },
    { "component": "zlib", "features": [] },
    { "component": "openssl", "features": [] }
  ]
}
```

```
{
  "subsystem": "developtools",
  "components": [
    { "component": "openssh", "features": [] }
  ]
}
```

注意： `zlib` 和 `openssl` 是 `openssh` 的编译依赖，必须同时注册到产品配置中。

2. OpenSSL 平台检测模块

新建文件： `third_party/openssl/openssl.gni`

从 `third_party/openssl/BUILD.gn` 中提取平台检测逻辑，创建独立的 `.gni` 文件，供 `openssh` 等依赖 OpenSSL 的组件引用：

```
import("//build/ohos.gni")

openssl_selected_platform = ""
if (current_cpu == "arm") {
  openssl_selected_platform = "linux-armv4"
} else if (current_cpu == "arm64") {
  openssl_selected_platform = "linux-aarch64"
} else if (current_cpu == "x86") {
  openssl_selected_platform = "linux-x86"
} else if (current_cpu == "x64") {
  openssl_selected_platform = "linux-x86_64"
}
```

用途： openssl 的 BUILD.gn 通过 `import("//third_party/openssl/openssl.gni")` 引入此变量，动态定位 OpenSSL 3.0.7 生成的头文件路径，避免硬编码。

3. zlib 适配系统类型修复

文件： `third_party/zlib/bundle.json`

zlib 的 `adapted_system_type` 原本只包含 `["mini", "standard"]`，缺少 `"small"`，导致 hi3519dv500 (small system) 编译时无法识别 zlib 组件。

修改前：

```
"adapted_system_type": ["mini", "standard"]
```

修改后：

```
"adapted_system_type": ["mini", "small", "standard"]
```

4. 构建脚本重写

文件： `developtools/openssl/BUILD.gn`

基于参考实现重写，主要变更如下：

4.1 引入 OpenSSL 平台检测

```
import("//third_party/openssl/openssl.gni")
```

4.2 依赖项定义

```
common_deps = [
  "//third_party/zlib:shared_libz",
  "//third_party/openssl:libcrypto_shared",
  "//third_party/openssl:libssl_shared",
]
```

4.3 编译选项调整

移除 ARM32 兼容宏 (hi3519dv500 为 ARM64) , 添加 `-DHAVE_CLOSEFROM` :

```
common_cflags_c = [
    "-D_BSD_SOURCE",
    "-D_DEFAULT_SOURCE",
    "-DHAVE_CONFIG_H",
    "-D_LIBEXEC_DIR=\"/bin\"",
    "-DHAVE_CLOSEFROM",
]
```

`-DHAVE_CLOSEFROM` 说明: musl libc 提供了 `int closefrom(int)` 函数, 其返回值类型与 openssl openssl-compat 中的 `void closefrom(int)` 不同。添加此宏让 openssl 使用系统自带的 closefrom, 避免类型冲突。

4.4 OpenSSL 头文件路径动态化

移除 `common_include_dirs` 中硬编码的 `ohos_lite/include` 路径 (该路径包含 OpenSSL 1.1.1n 旧版头文件, 与运行时 3.0.7 版本不匹配), 改为在 `libssh` 和 `libopenbsd-compat` 静态库中动态引用:

```
ohos_static_library("libssh") {
    include_dirs = common_include_dirs
    include_dirs += [ "../third_party/openssl/include" ]
    if (openssl_selected_platform != "") {
        include_dirs += [
            "${root_out_dir}/obj/third_party/openssl/build_all_generated/${openssl_selected_platform}/include" ]
    } else {
        assert(false, "ERROR: openssl_selected_platform is empty!")
    }
    # ...
}
```

关键点: 编译时头文件版本必须与运行时共享库版本一致。OpenSSL 1.1.1n 与 3.0.7 的 API 不兼容, 混用会导致 `OpenSSH version mismatch` 运行时错误。

4.5 Sandbox 后端选择

```
openssh_sandbox_backend = "rlimit"
```

选择 `rlimit` (POSIX 资源限制) 而非 `seccomp-filter` (Linux seccomp-bpf) , 原因:

- `seccomp-filter` 依赖 `linux/seccomp.h` 和 `linux/filter.h` , 在 OHOS musl libc 环境下存在头文件冲突
- `rlimit` 是跨平台 POSIX 方案, 稳定可靠

sshd-session 根据此变量选择 sandbox 实现并添加对应宏:

```

if (openssh_sandbox_backend == "rlimit") {
    sources += [ "sandbox-rlimit.c" ]
} else {
    sources += [ "sandbox-null.c" ]
}

if (openssh_sandbox_backend == "rlimit") {
    cflags_c += [ "-DSANDBOX_RLIMIT" ]
} else {
    cflags_c += [ "-DSANDBOX_NULL" ]
}

```

5. 源码头文件修复

5.1 openbsd-compat/port-net.c

文件： `developtools/openssh/openbsd-compat/port-net.c`

修改 1： 添加 `<sys/socket.h>` 包含（在 `<sys/ioctl.h>` 之后）：

```

#include <sys/ioctl.h>
#include <sys/socket.h>    /* 新增：确保 sockaddr 等类型定义 */

```

修改 2： 将 `<linux/if.h>` 替换为 `<net/if.h>`：

```

// 修改前
#include <linux/if.h>

// 修改后
#include <net/if.h>

```

原因： `<linux/if.h>` 与 musl libc 的 `<net/if.h>` 存在结构体重定义冲突，使用 `<net/if.h>` 更兼容。

5.2 sandbox-seccomp-filter.c

文件： `developtools/openssh/sandbox-seccomp-filter.c`

虽然最终选择了 `rlimit` 后端（不编译此文件），但仍需修复以便将来切换到 `seccomp-filter`：

修改 1： 头文件替换：

```

// 修改前
#include <linux/net.h>

// 修改后
#include <sys/socket.h>

```

修改 2： `ssh_sandbox_init()` 函数添加未使用参数标记：

```
static void
ssh_sandbox_init(struct monitor *monitor)
{
    (void)monitor; /* 新增：消除未使用参数警告 */
}
```

修改 3： `ssh_sandbox_child()` 函数简化，移除 `seccomp-bpf` 安装逻辑（因为 OHOS 环境下 `PR_SET_NO_NEW_PRIVS` 和 `seccomp` 系统调用支持不完整）：

```
static void
ssh_sandbox_child(struct ssh_sandbox *box)
{
    (void)box; /* 新增 */
    struct rlimit rl_zero;

    rl_zero.rlim_cur = rl_zero.rlim_max = 0;

    /* 移除了 nnp_failed 逻辑和 seccomp filter 安装代码 */
    debug3("%s: seccomp filter program is disabled on this port", __func__);
}
```

6. config.h 运行时路径修复

文件： `developtools/openssh/config.h`

6.1 USER_PATH 硬编码路径清理

修改前（第 1923 行）：

```
#define USER_PATH "/usr/bin:/bin:/usr/sbin:/sbin:/home/ubuntu/tools/aarch64-v01c01-linux-musl/openssh-8.0p1/bin"
```

修改后：

```
#define USER_PATH "/usr/bin:/bin:/usr/sbin:/sbin"
```

原因： `USER_PATH` 被编译进 `sshd` 二进制，作为 SSH 会话的默认 `$PATH` 环境变量。原值包含构建机上的绝对路径，在设备上不存在，导致 SSH 登录后 `PATH` 设置异常，甚至触发 Signal 11（SIGSEGV）崩溃。

7. session.c 空目录防护

文件： `developtools/openssh/session.c`

7.1 HOME 环境变量防护（第 1017 行）

修改前：

```
child_set_env(&env, &envsize, "HOME", pw->pw_dir);
```

修改后：

```
child_set_env(&env, &envsize, "HOME", pw->pw_dir[0] ? pw->pw_dir : "/");
```

原因：当 `/etc/passwd` 中 root 的 `pw_dir` 为空字符串时，`HOME=""` 会导致 SSH 会话异常甚至 Signal 11。防护逻辑：空则回退到 `/`。

7.2 chdir 防护（第 1613 行）

修改前：

```
if (chdir(pw->pw_dir) == -1) {
```

修改后：

```
if (chdir(pw->pw_dir[0] ? pw->pw_dir : "/") == -1) {
```

原因：同上，`chdir("")` 会失败并产生警告。防护逻辑与 HOME 一致。

8. SSH 配置文件修改

文件： `developtools/openssh/sshd_config`

将硬编码的构建机路径替换为设备标准路径：

```
- #Path /root/linux/tool/tar_tool/openssh/9.9p1/bin
- PATH=/root/linux/tool/tar_tool/openssh/9.9p1/bin:/usr/bin:/bin:/usr/sbin:/sbin
+ PATH=/usr/bin:/bin:/usr/sbin:/sbin

- #HostKey /root/linux/tool/tar_tool/openssh/9.9p1/etc/ssh_host_rsa_key
+ #HostKey /etc/ssh/ssh_host_rsa_key

- #HostKey /root/linux/tool/tar_tool/openssh/9.9p1/etc/ssh_host_ecdsa_key
+ #HostKey /etc/ssh/ssh_host_ecdsa_key

- #HostKey /root/linux/tool/tar_tool/openssh/9.9p1/etc/ssh_host_ed25519_key
+ #HostKey /etc/ssh/ssh_host_ed25519_key
```

9. SSH 启动脚本

新建文件： `vendor/hisilicon/hispark_hi3519dv500_linux/init_configs/etc/init.d/s85ssh`

最终版本的启动脚本如下（包含所有运行时修复）：

```
#!/bin/sh

echo "start ssh"
mkdir -p /var/log/
mkdir -p /var/empty
mkdir -p /etc/ssh/

busybox_path=/bin/busybox
if [ -f /usr/bin/busybox ];then
```

```

busybox_path=/usr/bin/busybox
echo "find busybox_path at ${busybox_path}"
fi
if [ -f $busybox_path ];then
echo "find busybox_path at ${busybox_path}"
if [ ! -L /bin/syslogd ];then
ln -s ${busybox_path} /bin/syslogd
fi
if [ ! -L /bin/klogd ];then
ln -s ${busybox_path} /bin/klogd
fi
if [ -L /bin/passwd ];then
echo "unlink /bin/passwd"
unlink /bin/passwd
fi
ln -s ${busybox_path} /bin/passwd
if [ -L /bin/ping ];then
echo "unlink /bin/ping"
unlink /bin/ping
fi
ln -s ${busybox_path} /bin/ping
if [ -L /bin/grep ];then
echo "unlink /bin/grep"
unlink /bin/grep
fi
ln -s ${busybox_path} /bin/grep
if [ -L /bin/chpasswd ];then
echo "unlink /bin/chpasswd"
unlink /bin/chpasswd
fi
ln -s ${busybox_path} /bin/chpasswd
fi

syslogd
klogd

# 创建 sshd 用户（OpenSSH 权限分离必需）
grep -q sshd /etc/passwd || echo "sshd:x:74:74::/var/empty:/bin/false" >> /etc/passwd
grep -q sshd /etc/group || echo "sshd:x:74:" >> /etc/group

# 修复 root shell（原为 /bin/false，SSH 登录需要有效 shell）
sed -i 's|root:x:0:0:::/bin/false|root:x:0:0:::/bin/sh|' /etc/passwd
mkdir -p /root

# 设置 root 密码（默认 123456，生产环境请修改）
echo "root:123456" | chpasswd

[ -f /etc/ssh/ssh_host_rsa_key ] || ssh-keygen -t rsa -f /etc/ssh/ssh_host_rsa_key -N ""

/bin/sshd -f /etc/sshd_config -h /etc/ssh/ssh_host_rsa_key

```

功能说明：

- 自动检测 busybox 路径（兼容 `/bin/busybox` 和 `/usr/bin/busybox`）
- 通过 BusyBox 符号链接提供 `syslogd`、`klogd`、`passwd`、`ping`、`grep`、`chpasswd` 命令
- 所有软连接创建前先 unlink 旧链接，避免重复创建
- 启动系统日志守护进程
- 创建 sshd 系统用户（OpenSSH 权限分离必需）
- 修复 root 的 shell 为 `/bin/sh`、HOME 为 `/`
- 通过 `chpasswd` 设置 root 默认密码
- 首次启动时自动生成 RSA 主机密钥
- 以指定配置文件和密钥启动 sshd（二进制位于 `/bin/sshd`）

关键路径说明：

- sshd 二进制安装位置：`/bin/sshd`（不是 `/usr/sbin/sshd`，OpenHarmony small system 的 `ohos_executable` 默认安装到 `/bin/`）
- sshd_config 配置文件：`/etc/sshd_config`（由 `ohos_prebuilt_etc` 安装）
- 主机密钥：`/etc/ssh/ssh_host_rsa_key`（首次启动自动生成）

10. S82ohos 脚本修复

文件： `vendor/hisilicon/hispark_hi3519dv500_linux/init_configs/etc/init.d/S82ohos`

对已有的 S82ohos 脚本进行路径兼容性修复和 ping 组权限配置：

修改 1： 添加 ping 组权限放开（在 `chmod 666 /dev/mmz_userdev` 之前）：

```
echo 0 9999999 > /proc/sys/net/ipv4/ping_group_range
```

原因： 默认的 `ping_group_range` 限制只有 root 可以使用 ICMP ping。设置为 `0 9999999` 允许所有用户组使用 ping 命令。

修改 2： busybox 和 btools 路径自动检测（替换原硬编码路径）：

```
busybox_path=/bin/busybox
if [ -f /usr/bin/busybox ];then
    busybox_path=/usr/bin/busybox
fi

if [ -f /usr/bin/btools ];then
    ln -s /usr/bin/btools /bin/bspmd.1
elif [ -f /bin/btools ];then
    ln -s /bin/btools /bin/bspmd.1
fi

if [ -f $busybox_path ];then
    if [ ! -L /bin/awk ];then
        ln -s ${busybox_path} /bin/awk
    fi
    if [ ! -L /bin/ipaddr ];then
```

```
ln -s ${busybox_path} /bin/ipaddr
fi
if [ ! -L /bin/vi ];then
    ln -s ${busybox_path} /bin/vi
fi
fi
```

原因：hi3519dv500 上 busybox 实际位于 `/bin/busybox`（不是 `/usr/bin/busybox`），btools 路径也不固定。自动检测机制兼容两种目录结构。

11. 启动脚本注册

文件： `vendor/hisilicon/hispark_hi3519dv500_linux/init_configs/BUILD.gn`

在 `init_initd` target 的 sources 列表中添加 S85ssh：

```
copy("init_initd") {
    sources = [
        "etc/init.d/S00devs",
        "etc/init.d/S01mdev",
        "etc/init.d/S80network",
        "etc/init.d/S82ohos",
        "etc/init.d/rcS",
        "etc/init.d/S85ssh",    /* 新增 */
    ]
    outputs = [ "$root_out_dir/etc/init.d/{{source_file_part}}" ]
}
```

三、问题解决记录

问题 1: closefrom 返回值类型冲突（编译期）

错误信息：

```
error: conflicting types for closefrom
openbsd-compat/bsd-closefrom.c:30:6: note: previous definition is here
```

原因： musl libc 定义 `int closefrom(int)`，而 openssl openbsd-compat 定义 `void closefrom(int)`，返回值类型不同。

解决方案： 在 BUILD.gn 的 `common_cflags_c` 中添加 `-DHAVE_CLOSEFROM`，让 openssl 使用系统自带的 closefrom 而非自己实现。

问题 2: OpenSSL 版本不匹配（运行期）

错误信息（运行时）：

```
OpenSSH version mismatch
```

原因： 编译时使用了 `ohos_lite/include` 下的 OpenSSL 1.1.1n 头文件，而运行时链接的是 OpenSSL 3.0.7 共享库。两个大版本的 API 不兼容，导致版本校验失败。

解决方案:

1. 从 `common_include_dirs` 中移除 `ohos_lite/include`
2. 通过 `openssl.gni` 的 `openssl_selected_platform` 变量动态定位正确的 3.0.7 头文件路径

问题 3: zlib 组件未被识别 (编译期)

错误信息:

```
check_deps_handler.py exception: zlib part not found
```

原因: `third_party/zlib/bundle.json` 中 `adapted_system_type` 为 `["mini", "standard"]`, 不包含 `"small"`, 导致 hi3519dv500 (small system) 编译时无法匹配 zlib 组件。

解决方案: 在 `adapted_system_type` 中添加 `"small"`。

问题 4: sockaddr_storage 类型未定义 (编译期)

错误信息:

```
netinet/in.h: error: field gr_group has incomplete type struct sockaddr_storage
```

原因: musl libc 的 `netinet/in.h` 依赖 `struct sockaddr_storage`, 该类型定义在 `<linux/socket.h>` 中。当使用了 `-D_UAPI_LINUX_SOCKET_H` 宏时, 会跳过 `linux/socket.h` 的包含, 导致类型缺失。

解决方案: 在 `port-net.c` 中显式包含 `<sys/socket.h>`, 确保 `sockaddr_storage` 类型可用。

问题 5: linux/if.h 与 net/if.h 冲突 (编译期)

错误信息:

```
redefinition of struct ifmap  
redefinition of struct ifreq
```

原因: `<linux/if.h>` 和 musl libc 的 `<net/if.h>` 都定义了 `struct ifmap`、`struct ifreq` 等网络接口结构体, 同时包含会导致重定义错误。

解决方案: 将 `#include <linux/if.h>` 替换为 `#include <net/if.h>`。

问题 6: seccomp-filter 编译失败 (编译期)

错误信息:

```
linux/seccomp.h: No such file or directory  
linux/filter.h: No such file or directory
```

原因: OHOS musl libc 环境下缺少 `linux/seccomp.h` 和 `linux/filter.h` 头文件, 且 `linux/net.h` 与 musl 的 `sys/socket.h` 存在冲突。

解决方案: 将 sandbox 后端从 `seccomp-filter` 切换为 `rlimit`, 避免 `seccomp-bpf` 依赖。

问题 7：脚本执行权限被拒绝（编译期）

错误信息：

```
PermissionError: [Errno 13] Permission denied
```

原因：Windows/Linux 跨文件系统挂载，脚本文件默认没有 `+x` 权限。

解决方案：手动添加执行权限：

```
find developtools/openssh -type f | while read f; do head -c 20 "$f" 2>/dev/null | grep -qE '^#!' && chmod +x "$f"; done
```

问题 8：USER_PATH 包含构建机硬编码路径（运行期）

错误信息：SSH 登录后 Signal 11 (SIGSEGV) 崩溃。

原因：`config.h` 第 1923 行 `USER_PATH` 宏包含构建机的绝对路径 `/home/ubuntu/tools/aarch64-v01c01-linux-musl/openssh-8.0p1/bin`，该路径在设备上不存在。此值被编译进 `sshd` 二进制，作为 SSH 会话的默认 `$PATH`。

解决方案：将 `USER_PATH` 修改为标准路径：

```
#define USER_PATH "/usr/bin:/bin:/usr/sbin:/sbin"
```

问题 9：空 pw_dir 导致 SSH 会话崩溃（运行期）

错误信息：SSH 登录时 Signal 11 (SIGSEGV) 。

原因：`/etc/passwd` 中 root 条目的 `pw_dir` 为空字符串时：

- `HOME=""` 导致环境变量异常
- `chdir("")` 调用失败

解决方案：在 `session.c` 中添加空字符串防护：

```
// 第 1017 行 - HOME 防护
child_set_env(&env, &envsize, "HOME", pw->pw_dir[0] ? pw->pw_dir : "/");

// 第 1613 行 - chdir 防护
if (chdir(pw->pw_dir[0] ? pw->pw_dir : "/") == -1) {
```

问题 10：sshd 权限分离用户不存在（运行期）

错误信息：

```
privilege separation user sshd does not exist
```

原因：OpenSSH 的 privilege separation 功能要求 `/etc/passwd` 中存在 `sshd` 系统用户。设备默认的 `passwd` 文件中没有此用户。

解决方案： 在 S85ssh 启动脚本中添加：

```
grep -q sshd /etc/passwd || echo "sshd:x:74:74::/var/empty:/bin/false" >> /etc/passwd
grep -q sshd /etc/group || echo "sshd:x:74:" >> /etc/group
```

问题 11：root shell 为 /bin/false 导致 SSH 登录失败（运行期）

错误信息： SSH 连接后立即断开。

原因： 设备默认的 `/etc/passwd` 中 root 的 shell 为 `/bin/false`，SSH 登录后立即退出。

解决方案： 在 S85ssh 中通过 sed 修复：

```
sed -i 's|root:x:0:0:::/bin/false|root:x:0:0:::/bin/sh|' /etc/passwd
```

注意 passwd 格式： `/etc/passwd` 必须是 7 个字段（username:password:uid:gid:GECOS:home:shell，共 6 个冒号）。替换后的格式为 `root:x:0:0:::/bin/sh`，其中 GECOS 为空（两个连续冒号 `::`）、home 为 `/`、shell 为 `/bin/sh`。如果误写成 `root:x:0:0:/bin/sh`（只有 5 个冒号），shell 字段会丢失，导致 `passwd: /etc/passwd: bad record` 错误。

问题 12：chpasswd 命令不存在（运行期）

错误信息：

```
sh: chpasswd: inaccessible or not found
```

原因： BusyBox 编译时包含了 `chpasswd` applet，但 `/bin/` 下没有创建指向 busybox 的 `chpasswd` 软连接。

解决方案： 在 S85ssh 的 busybox 软连接块中添加 `chpasswd` 链接：

```
if [ -L /bin/chpasswd ];then
    echo "unlink /bin/chpasswd"
    unlink /bin/chpasswd
fi
ln -s ${busybox_path} /bin/chpasswd
```

问题 13：grep 报 Invalid argument（运行期）

错误信息：

```
grep: (standard input): Invalid argument
```

原因： 设备上 `/bin/grep` 不是指向 busybox 的软连接（可能是其他版本的 grep 或链接损坏），无法正确处理 `/proc` 文件系统的输入。

解决方案： 在 S85ssh 中重建 grep 软连接：

```
if [ -L /bin/grep ];then
    echo "unlink /bin/grep"
    unlink /bin/grep
fi
ln -s ${busybox_path} /bin/grep
```

问题 14：sshd 二进制路径错误（运行期）

错误信息： sshd 进程未启动，但无报错信息。

原因： S85ssh 中使用 `/usr/sbin/sshd` 启动 sshd，但 OpenHarmony small system 的 `ohos_executable` 默认将可执行文件安装到 `/bin/`，实际路径为 `/bin/sshd`。路径错误导致启动命令静默失败（`/usr/sbin/sshd` 不存在）。

解决方案： 将 S85ssh 中的 sshd 启动路径修改为：

```
/bin/sshd -f /etc/sshd_config -h /etc/ssh/ssh_host_rsa_key
```

验证方法： 在设备上执行 `ls /bin/sshd` 确认二进制位置。也可以用 `find / -name sshd -type f` 全盘搜索。

四、编译验证

编译命令

```
# 编译（推荐）
hb build developertools/openssh:ssh

# 全量编译
rm -rf out && hb build -f
```

编译结果

```
[OHOS INFO] build success
```

生成文件

OpenSSH 编译产生 7 个可执行文件和 1 个配置文件：

文件	设备路径	说明
ssh	/bin/ssh	SSH 客户端
sshd	/bin/sshd	SSH 服务端守护进程
sshd-session	/bin/sshd-session	SSH 会话处理进程
scp	/bin/scp	安全复制命令
sftp	/bin/sftp	SFTP 客户端

文件	设备路径	说明
sftp-server	/bin/sftp-server	SFTP 服务端
ssh-keygen	/bin/ssh-keygen	密钥生成工具
sshd_config	/etc/sshd_config	sshd 配置文件

输出路径: out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/

文件信息

```
$ file out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/bin/ssh
ssh: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV),
      dynamically linked, interpreter /lib/ld-musl-aarch64.so.1, stripped

$ file out/hispark_hi3519dv500/ipcamera_hispark_hi3519dv500_linux/bin/sshd
sshd: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV),
      dynamically linked, interpreter /lib/ld-musl-aarch64.so.1, stripped
```

文件特性:

- 架构: ARM 64-bit (aarch64)
- 链接方式: 动态链接
- 解释器: musl libc (/lib/ld-musl-aarch64.so.1)
- PIE: 位置无关可执行文件

五、功能说明

OpenSSH 9.9p1 提供以下核心功能:

远程登录

- ssh — 加密的远程终端登录
- 支持 RSA、ECDSA、Ed25519 密钥认证
- 支持密码认证和公钥认证

文件传输

- scp — 基于 SSH 协议的安全复制
- sftp — 交互式 SFTP 客户端
- sftp-server — SFTP 服务端子系统

密钥管理

- `ssh-keygen` — 生成、管理和转换 SSH 密钥
- 支持 RSA、ECDSA、Ed25519、XMSS 密钥类型

服务端

- `sshd` — SSH 守护进程，监听传入连接
- `sshd-session` — 处理单个 SSH 会话

加密算法

- 支持多种加密算法 (AES-CTR、AES-GCM、ChaCha20-Poly1305)
- 支持多种密钥交换算法 (curve25519-sha256、ECDH、DH-GEX)
- 支持多种 MAC 算法 (hmac-sha2-256、umac-128)

六、使用方法

1. 服务端启动

系统启动时，`s85ssh` 脚本会自动启动 `sshd`。也可以手动启动：

```
# 首次使用，生成主机密钥
ssh-keygen -t rsa -f /etc/ssh/ssh_host_rsa_key -N ""

# 启动 sshd
/bin/sshd -f /etc/sshd_config -h /etc/ssh/ssh_host_rsa_key
```

2. 客户端连接

```
# 从其他设备连接到 Hi3519DV500
ssh root@<device_ip>
# 默认密码：123456

# 使用密钥认证
ssh -i /path/to/private_key root@<device_ip>
```

3. 文件传输

```
# 使用 scp 复制文件
scp file.txt root@<device_ip>:/tmp/

# 使用 sftp 交互式传输
sftp root@<device_ip>
```

4. 密钥管理

```
# 生成 RSA 密钥对
ssh-keygen -t rsa -b 4096

# 生成 Ed25519 密钥对
ssh-keygen -t ed25519
```

七、与其他组件的关系

依赖关系

```
openssh
├─ openssl (3.0.7)    - 加密库 (libcrypto, libssl)
├─ zlib (1.2.13)      - 压缩库
├─ musl libc          - C 标准库
├─ busybox            - 提供运行时工具 (passwd, chpasswd, grep, syslogd 等)
└─ linux kernel       - 系统调用、网络栈
```

配合使用

1. 与 BusyBox 配合：

- S85ssh 脚本通过 BusyBox 符号链接提供 `syslogd`、`klogd`、`passwd`、`ping`、`grep`、`chpasswd` 命令
- BusyBox 的 `passwd` 命令用于设置用户密码，`chpasswd` 用于脚本中批量设置密码
- BusyBox 的 `grep` 替代原有有缺陷的 `grep`，修复 `Invalid argument` 问题
- 脚本自动检测 busybox 路径（`/bin/busybox` 或 `/usr/bin/busybox`），兼容不同目录结构

2. 与 OpenSSL 配合：

- 编译时依赖 OpenSSL 3.0.7 的头文件和静态库
- 运行时依赖 `libcrypto_shared` 和 `libssl_shared` 共享库
- **版本必须一致**：编译时头文件版本与运行时库版本不匹配会导致 `OpenSSH version mismatch` 错误

3. 与 iptables 配合：

- 可通过 iptables 规则限制 SSH 访问来源
- 示例：`iptables -A INPUT -p tcp --dport 22 -s 192.168.1.0/24 -j ACCEPT`

八、注意事项

1. OpenSSL 版本一致性

这是最关键的注意事项。编译时使用的 OpenSSL 头文件版本必须与运行时共享库版本一致。本项目使用 OpenSSL 3.0.7，头文件路径通过 `openssl_selected_platform` 变量动态定位。如果误用了旧版（1.1.1n）头文件，编译能通过但运行时会报 `OpenSSH version mismatch` 错误。

2. 架构差异

本产品 (hispark_hi3519dv500_linux) 使用 ARM64 架构:

- 编译工具链: `aarch64-v01c01-linux-musl-gcc`
- 动态链接器: `/lib/ld-musl-aarch64.so.1`
- `openssl_selected_platform`: `linux-aarch64`

与 hispark_taurus_linux (ARM 32-bit) 的差异:

- 工具链: `arm-v01c01-linux-musleabi-gcc`
- `openssl_selected_platform`: `linux-armv4`
- 部分编译宏不同

3. Sandbox 后端选择

当前使用 `rlimit` sandbox 后端。如果未来需要切换到 `seccomp-filter`:

- 需要修复 `sandbox-seccomp-filter.c` 中的头文件依赖问题
- 需要确认内核支持 `CONFIG_SECCOMP` 和 `CONFIG_SECCOMP_FILTER`
- `PR_SET_NO_NEW_PRIVS` 在 OHOS 环境下可能需要额外适配

4. 运行时依赖

SSH 服务运行需要以下条件:

- `/var/empty` 目录 (sshd 权限分离 chroot 目录)
- `/etc/ssh/` 目录 (存放主机密钥)
- `/etc/passwd` 中需要有 `sshd` 用户条目 (权限分离模式)
- `/var/log/` 目录 (syslogd 日志目录)
- BusyBox 提供的 `passwd`、`chpasswd`、`grep`、`syslogd`、`klogd` 软连接

5. 脚本权限

通过 Windows 挂载的代码目录, 脚本文件需要手动添加执行权限。建议在构建前执行权限修复。

6. 安全考虑

- 默认密码为 `123456`, 生产环境必须修改
- 建议修改默认的 `sshd_config`, 禁用不安全的加密算法
- 生产环境中建议禁用密码认证, 仅使用公钥认证
- 建议修改默认 SSH 端口 (22)
- 确保 `/etc/ssh/` 目录和密钥文件权限正确 (密钥文件应为 600)

7. 设备上二进制文件位置

OpenHarmony small system 中，ohos_executable 默认将可执行文件安装到 /bin/ 目录（不是 /usr/bin/ 或 /usr/sbin/）。因此所有 OpenSSH 工具 (ssh、sshd、scp、sftp 等) 都在 /bin/ 下。如果脚本中使用了 /usr/sbin/sshd 等路径，需要改为 /bin/sshd。

8. /etc/passwd 格式要求

修改 /etc/passwd 时必须保持 7 个字段格式 (username:password:uid:gid:GECOS:home:shell，即 6 个冒号分隔符)。常见错误是漏掉 GECOS 字段的空值冒号，例如将 root:x:0:0:::/bin/sh 误写为 root:x:0:0::/bin/sh（少了一个冒号），导致 shell 字段丢失，busybox passwd 报 bad record 错误。

九、修改文件清单

文件	操作	说明
vendor/.../config.json	修改	添加 zlib、openssl、openssh 组件注册
vendor/.../init_configs/BUILD.gn	修改	添加 S85ssh 到 init_initd
vendor/.../init_configs/etc/init.d/s85ssh	新建	SSH 服务启动脚本（含 busybox 链接、用户创建、密码设置）
vendor/.../init_configs/etc/init.d/s82ohos	修改	busybox/btools 路径检测 + ping_group_range
third_party/openssl/openssl.gni	新建	OpenSSL 平台检测模块
third_party/zlib/bundle.json	修改	adapted_system_type 添加 "small"
developtools/openssh/BUILD.gn	重写	重写构建脚本，适配 ARM64
developtools/openssh/config.h	修改	USER_PATH 去除硬编码构建机路径
developtools/openssh/session.c	修改	HOME 环境变量和 chdir 的空 pw_dir 防护
developtools/openssh/openssh-compat/port-net.c	修改	头文件修复 (sys/socket.h, net/if.h)
developtools/openssh/sandbox-seccomp-filter.c	修改	头文件和函数修复（备用，当前未编译）
developtools/openssh/sshd_config	修改	路径替换为设备标准路径

注： `vendor/...` 为 `vendor/hisilicon/hispark_hi3519dv500_linux/` 的简写。

十、参考资料

- OpenSSH 官网: <https://www.openssh.com/>
- OpenSSH 源码: <https://github.com/openssh/openssh-portable>
- OpenSSL 官网: <https://www.openssl.org/>
- OpenHarmony 文档: <https://gitee.com/openharmony>
- GN 构建系统: <https://gn.googlesource.com/gn/>